Full Stack Developer Intern – 4 Days Assignment

MindX Digital Softwares

Objective (Please Read Carefully)

In this assignment, you will build a **mini working version of MindX Service AI**.

Expected Flow:

1. Customer sends a query
2. AI generates a response
3. A support ticket is created
4. Admin can view and manage the ticket

This is a **real-world product simulation**, not a theoretical task.
We are evaluating you as a **potential core developer**, not just an intern.

Tech Stack (Mandatory)

Backend (Required)

- Java
- Spring Boot
- REST APIs
- JPA / Hibernate

Frontend

- React.js

Database

- MySQL / PostgreSQL

Day 1 – Backend Setup & Core APIs

Goal:

Set up backend system and basic APIs.

Step 1: Create Spring Boot Project

Use Spring Initializr and include:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Lombok (optional)

Step 2: Database Design

Create the following entities:

1. User

- id
- name

2. Ticket

- id
- userId
- query
- status
    - OPEN
    - RESOLVED
    - NEEDS_HUMAN
- createdAt

3. Message

- id
- ticketId
- sender
    - USER
    - AI
    - ADMIN
- message
- timestamp

Step 3: Build APIs

Create REST APIs:

1. Create Ticket

POST /tickets

Input:

- query

Output:

- Ticket created

2. Get All Tickets

GET /tickets

Output:

- List of tickets

3. Get Ticket by ID

GET /tickets/{id}

Output:

- Ticket details
- All messages for that ticket

Day 2 – AI Integration + Core Logic

Goal:

Add AI-based response system.

Step 1: AI Integration

You can use:

- OpenAI API (preferred)
  OR
- Mock AI response (if API not available)

Step 2: Core Logic Flow

When user sends a query:

1. Create ticket
2. Store user message
3. Call AI API
4. Get AI response
5. Store AI response
6. Return response to frontend

Example:

User: “Where is my order?”
AI: “Please share your order ID to assist you further.”

Both messages must be stored in database.

Day 3 – Frontend (React Chat UI)

Goal:

Build customer-facing chat interface.

Step 1: Chat Interface

Create a simple UI with:

- Input box
- Send button
- Chat message display

Step 2: Integration

- On clicking “Send”:
  - Call backend API
  - Display:
    - User message
    - AI response

UI Expectations:

- Clean layout
- Readable messages
- No need for heavy design

Day 4 – Admin Dashboard + Smart Logic

Goal:

Complete system with admin control and improvements.

Step 1: Admin Dashboard

Ticket List Page

Display:

- Query
- Status
- Created time

Ticket Detail Page

Display:

- Full conversation
- Messages (user + AI)

Status Update

Admin can change:

- OPEN
- RESOLVED
- NEEDS_HUMAN

Step 2: Escalation Logic

If user query contains keywords like:

- “refund”
- “complaint”
- “angry”

Automatically set:
status = NEEDS_HUMAN

Step 3: Improvements

- Prevent empty messages
- Handle API errors
- Clean UI layout
- Basic responsiveness

Final Submission

You must submit:

1. GitHub Repository

- Well-structured code
- Proper folder organization

2. README File

Include:

- Tech stack used
- Setup instructions
- Features implemented

3. Demo (Optional but Preferred)

- Video recording OR
- Live hosted link

Bonus (Strong Advantage)

If you implement any of these:

- Login / authentication
- Real-time chat (WebSockets)
- Analytics dashboard
- Better UI/UX
- Proper layered architecture:
    - Controller
    - Service
    - Repository

Evaluation Criteria

◇ Technical Skills

- Spring Boot structure
- API design
- Database design

◇ Code Quality

- Clean, readable code
- Proper naming
- Separation of layers

◇ Product Thinking

- Does it feel like a real support system?

◇ Ownership

- Completed fully
- Extra effort beyond basics

◇ Speed & Execution

- Time management
- Prioritization

Important Note

This is not a basic intern assignment.

We are looking for someone who can:

- Build real SaaS features
- Take ownership
- Learn fast
- Replace a **core developer role**

Final Advice

Keep it simple
Make it working
Focus on backend quality

Working product > Perfect code

All the best